Traditional (Pre-GNN) Approaches to Machine Learning on Graphs

Dr. Srinivasa L. Chakravarthy
chakri.ls@wilp.bits-pilani.ac.in

AIML ZG514: Graph Neural Networks

BITS Pilani
02 August 2026

Outline of presentation

1. Where pre-GNN features fit in the pipeline
2. Importance-based features (centrality measures)
3. Structure-based features
4. Preliminaries: the 1-WL algorithm
5. Graph kernels
6. Neighborhood overlap detection (link prediction)

2 Where pre-GNN features fit in the pipeline

The traditional, two-stage pipeline

Before GNNs, machine learning on graph data followed a **two-stage** pipeline:

1. **Extract** hand-engineered features from the graph
   node-level statistics, *graph-level* summaries, or *pairwise* overlap scores.
2. **Feed** those features to a standard classifier
   logistic regression, SVM, random forest, gradient boosting, . . .

This session develops the **feature-extraction** side of the pipeline, organized by the prediction task it supports:

Task	**Granularity**	**Features today**
Node classification	per-node	importance- and structure-based
Graph classification	per-graph	bag-of-nodes, WL kernel, graphlet kernel
Link / relation prediction	per-pair	neighborhood-overlap measures

Why study it now? The Weisfeiler–Leman algorithm and the graph Laplacian (next session) are the direct ancestors of the GNN architectures of Part III; local-overlap heuristics remain competitive baselines for link prediction even today.

Two flavours of node-level features

At a high level, node-level features split into two camps:

Importance-based features

How influential / central is the node?

- Node degree
- Eigenvector centrality
- Katz centrality
- PageRank
- Closeness centrality
- Betweenness centrality

Structure-based features

What does the local neighborhood look like?

- Node degree
- Clustering coefficient
- Graphlet (degree) count vector

Note. Node degree appears in both columns: read as a *count of neighbors* it is a rudimentary importance measure; read as a *description of the immediate neighborhood* it is the simplest structural descriptor.

3 Importance-based features (centrality measures)

Why centrality measures?

Issue. Small networks can be visualized; large networks cannot. We need quantitative summaries that capture interesting aspects of network structure.

Centrality measures identify which nodes are *important* (central, influential) in a network.

The catch: there is no single "correct" notion of importance. Each measure formalizes a different intuition:

Measure	**Intuition**
Degree	"How many friends do I have?"
Eigenvector	"*Who* I know matters, not just how many."
Katz	Eigenvector + a free baseline (works on DAGs).
PageRank	Katz + divide by out-degree.
Closeness	"How quickly can I reach everyone?"
Betweenness	"How often does information flow through me?"

Source: Newman, Networks, Chapter 7 (2018).

3 Importance-based features (centrality measures)

3.1 Degree centrality

Degree centrality

Definition. The degree centrality (a.k.a. degree) of a node is the *number of its neighbors*.

A simple but useful measure of importance.

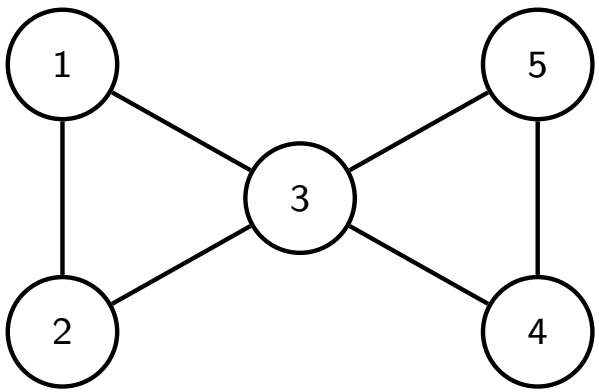
Examples in practice:

- Number of friends in a social network (Facebook, LinkedIn) measures a user's influence.
- In a citation network ($x \to y$ if x cites y), *in-degree* measures how influential a paper is.

Terminology. In social-network analysis, "degree centrality" (rather than just "degree") signals that the quantity is being used as a centrality measure.

Example: degree centrality on the bowtie graph

Bowtie: two triangles glued at the central vertex 3.



Neighborhoods:

- $N(1) = \{2, 3\}$, $N(2) = \{1, 3\}$
- $N(3) = \{1, 2, 4, 5\}$
- $N(4) = \{3, 5\}$, $N(5) = \{3, 4\}$

Degree centralities:

$$d_1 = 2,\ d_2 = 2,\ \boxed{d_3 = 4},\ d_4 = 2,\ d_5 = 2.$$

The hub 3 is the most central; the four leaves $\{1, 2, 4, 5\}$ are tied.

Foreshadowing. Later we'll see that the *local clustering coefficient* assigns node 3 the *lowest* score among the five vertices — the opposite of degree. The two features are genuinely complementary.

3 Importance-based features (centrality measures)

3.2 Eigenvector centrality

Why eigenvector centrality?

Issue with degree. Each neighbor contributes 1 to the count, even if that neighbor is unimportant.

What we'd really like: *a node is important if its neighbors are important*.

If you know only one person, but that person is the prime minister, you are still important.

Recursive definition (circular but solvable). Let $G = (V, E)$, $V = [n]$, A adjacency. Define

$$x_i = \frac{1}{\kappa} \sum_{j:\, j \sim i} x_j = \frac{1}{\kappa} \sum_{j=1}^{n} A_{ij}\, x_j \quad \Longleftrightarrow \quad A\,x = \kappa\, x.$$

The vector x of centrality scores is an *eigenvector* of A. Which one? The Perron–Frobenius theorem picks out the unique eigenvector with all nonnegative entries: the principal one.

Eigenvector centrality

Definition. Let $G = (V, E)$ be a connected graph with $V = [n]$, and let A be its adjacency matrix. The *leading* or *principal eigenvector* of A is the unit-length, nonnegative eigenvector corresponding to the eigenvalue of largest modulus. The eigenvector centrality of node $i \in V$ is the i-th component of this eigenvector.

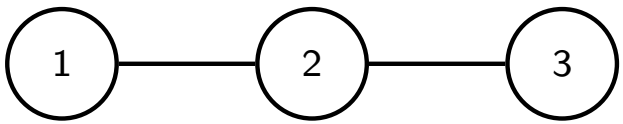
Why the principal eigenvector? By Perron–Frobenius (applicable since G is connected, so A is irreducible), the eigenvalue of largest modulus is positive and equals the spectral radius $\rho(A)$. The corresponding eigenvector has three properties no other eigenvector has:

- It is the *only* eigenvector of A with all entries nonnegative. Every non-principal eigenvector has both signs.
- It is canonical (no ambiguity about which one to pick), whereas there are n eigenvectors otherwise.
- It preserves the recursive semantics: more connections to important people *help*; connections to unimportant people *help less* — but never *hurt*.

Disconnected graphs. Run Perron–Frobenius per component; scores across components are then incomparable.

Example: eigenvector centrality on P_3

Path on three vertices, $V = \{1, 2, 3\}$, $E = \{\{1, 2\}, \{2, 3\}\}$.



$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

Char. poly. $\det(A - \lambda I) = -\lambda(\lambda^2 - 2)$, so $\lambda \in \{-\sqrt{2},\, 0,\, +\sqrt{2}\}$.

Principal eigenvalue: $\lambda_1 = \sqrt{2}$.

Solve $A\mathbf{x} = \sqrt{2}\,\mathbf{x}$. The three rows give

$$x_2 = \sqrt{2}\,x_1, \qquad x_1 + x_3 = \sqrt{2}\,x_2, \qquad x_2 = \sqrt{2}\,x_3,$$

forcing $x_1 = x_3$. Normalizing to unit length,

$$\mathbf{x} = \tfrac{1}{2}\,(1,\ \sqrt{2},\ 1)^\top \approx (0.500,\ 0.707,\ 0.500)^\top.$$

The middle node 2 is more central than the endpoints — exactly as the recursive intuition predicts. All scores are strictly positive, as Perron–Frobenius guarantees.

3 Importance-based features (centrality measures)

3.3 Katz centrality

Adjacency matrix of a directed graph

The Katz, PageRank, and centrality measures we develop next all work on directed graphs. We need a matrix representation.

Definition. Let $G = (V, E)$ be a directed graph with $V = [n]$. The adjacency matrix of G is $A \in \{0,1\}^{n \times n}$ with

$$A_{ij} = \begin{cases} 1, & \text{if } i \to j \text{ is an arc of } G, \\ 0, & \text{otherwise.} \end{cases}$$

Reading off A.

- Row i of A lists the *out*-neighbors of i: row sum $= d_i^{\text{out}}$.
- Column i of A lists the *in*-neighbors of i: column sum $= d_i^{\text{in}}$.
- For undirected graphs (treat each edge as two arcs), $A = A^\top$ and the two notions coincide.

Aggregating over in-neighbors. Many centrality recursions ask: "sum some quantity over the in-neighbors of i." Under our convention,

$$\sum_{j:\, j \to i} x_j = \sum_{j=1}^{n} A_{ji}\, x_j = \left(A^\top x\right)_i.$$

So in-neighbor aggregation is the matrix–vector product $A^\top x$, not Ax.

Why Katz centrality? Eigenvector centrality fails on DAGs

On a directed graph, eigenvector centrality says i is important if its in-neighbors are important:

$$x_i \;\propto\; \sum_{j:\, j\to i} x_j \;=\; \sum_{j} A_{ji}\, x_j, \qquad \text{i.e.,} \qquad A^\top x \;=\; \kappa\, x.$$

The principal eigenvector of $A^\top$ gives the scores.

Issue. If u has no in-neighbors then $x_u = 0$; similarly if $u \to v$ is v's only in-edge then $x_v = 0$. By induction, on a DAG the recursion drives most scores to zero — only sinks survive, and they sit at eigenvalue 0 rather than the positive principal eigenvalue Perron–Frobenius would otherwise guarantee. The framework becomes degenerate.

Citation networks are DAGs (x cites $y \Rightarrow x$ is published *after* y), so eigenvector centrality is useless here.

Fix. Give each node a "free" baseline β and scale the recursive term by α:

$$x \;=\; \alpha\, A^\top x + \beta\, \mathbf{1} \quad\Longrightarrow\quad x \;=\; \beta\,(I - \alpha A^\top)^{-1}\,\mathbf{1},$$

for $\alpha > 0$ small enough that $(I - \alpha A^\top)$ is invertible. Since only relative scores matter, set $\beta = 1$.

Katz centrality

Definition. Let G be a directed graph with adjacency matrix A. The Katz centrality scores of the nodes are

$$\boxed{x \;=\; (I - \alpha\, A^{\top})^{-1}\, \mathbf{1}.}$$

What does this mean? Use the Neumann series

$$(I - \alpha A^{\top})^{-1} \;=\; \sum_{k=0}^{\infty} \alpha^{k} (A^{\top})^{k}$$

(valid for $|\alpha|$ smaller than $1/\rho(A)$; note $\rho(A^{\top}) = \rho(A)$). Reading off:

$$x_i \;=\; \sum_{k=0}^{\infty} \alpha^{k} \left((A^{\top})^{k}\, \mathbf{1}\right)_i \;=\; \sum_{k=0}^{\infty} \alpha^{k} \cdot \#\{\text{walks of length } k \text{ ending at } i\}.$$

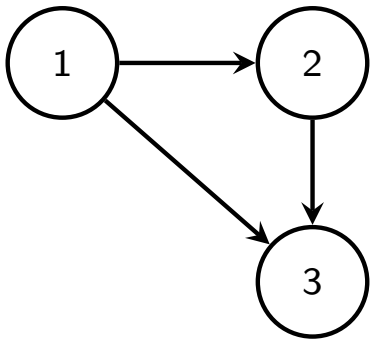
(Under our convention, $(A^{\top})^{k}_{ij}$ counts length-k walks from j to i, so $(A^{\top})^{k}\mathbf{1}$ at position i counts walks ending at i.)

A node is important if many walks of every length end at it, with shorter walks weighted more (factor α^{k}).

Hyperparameter α. Must be chosen below $1/\rho(A)$ for convergence; see Newman (2018, p. 164) for guidance.

Example: Katz centrality on a 3-node DAG (1/3)

Three vertices, arcs $1 \to 2,\ 1 \to 3,\ 2 \to 3$ (citation-like).



With $A_{ij} = 1$ iff $i \to j$ (CLRS convention):

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}, \qquad A^\top = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}.$$

A is nilpotent ($A^3 = 0$), spectrum $\{0, 0, 0\}$.

Eigenvector centrality is degenerate, not undefined. The principal eigenvalue is 0, and solving $A^\top x = 0$ gives $x = (0, 0, 1)^\top$: only the sink (node 3) scores nonzero. On DAGs with multiple sinks the null space becomes multi-dimensional and Perron–Frobenius uniqueness breaks — there is no longer a canonical eigenvector to call "the" centrality.

Katz to the rescue. Choose $\alpha = \frac{1}{2}$ (any $\alpha > 0$ works, since $A^\top$ is nilpotent). The Neumann series

$$(I - \alpha A^\top)^{-1} = \sum_{k=0}^{\infty} \alpha^k (A^\top)^k$$

collapses to a finite sum, since $(A^\top)^k = 0$ for $k \geq 3$. We expand it by hand on the next slide.

Example: Katz centrality on a 3-node DAG (2/3)

With $\alpha = \frac{1}{2}$ and $(A^\top)^k = 0$ for $k \geq 3$, the Neumann series collapses to three terms:

$$(I - \tfrac{1}{2} A^\top)^{-1} \;=\; I \;+\; \tfrac{1}{2} A^\top \;+\; \tfrac{1}{4} (A^\top)^2.$$

Compute the squared term:

$$(A^\top)^2 \;=\; \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix} \;=\; \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}.$$

Add the three weighted matrices:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 \\ \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{1}{4} & 0 & 0 \end{pmatrix} \;=\; \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{3}{4} & \frac{1}{2} & 1 \end{pmatrix}.$$

Multiply by $\mathbf{1}$ (sum the rows):

$$x \;=\; (I - \tfrac{1}{2} A^\top)^{-1} \mathbf{1} \;=\; \left(1,\; \tfrac{3}{2},\; \tfrac{9}{4}\right)^\top.$$

Example: Katz centrality on a 3-node DAG (3/3)

The score vector $x = (1,\ \frac{3}{2},\ \frac{9}{4})^\top$ makes structural sense.

Walk-counting interpretation. Each x_i counts walks ending at i, weighted by $\alpha^k = (\frac{1}{2})^k$:

- **Node** 1 has no in-walks, only the length-0 walk $\Rightarrow x_1 = 1$ (just the baseline).
- **Node** 2: length-0 walk, plus one length-1 walk $1 \to 2 \Rightarrow x_2 = 1 + \frac{1}{2} = \frac{3}{2}$.
- **Node** 3: length-0, two length-1 walks ($1 \to 3$ and $2 \to 3$), and one length-2 walk $1 \to 2 \to 3 \Rightarrow x_3 = 1 + 1 + \frac{1}{4} = \frac{9}{4}$.

Why this matches intuition. Node 3 wins on both quantity (more in-walks) and structure (the only node reachable by a length-2 walk). Upstream node 1 gets only the free baseline. Node 2 sits in between — one citation upstream, one downstream from it.

The same DAG pattern appears in citation networks (arc = "cites") and software dependency graphs (arc = "depends on") — both cases where eigenvector centrality breaks and Katz is the standard fix.

3 Importance-based features (centrality measures)

3.4 PageRank

Why PageRank? An issue with Katz

Katz's recursion: a node's score is proportional to the sum of the scores of its in-neighbors.

Pathological case. Suppose `amazon.com` is very important (high Katz score) and links to one million other pages.

Under Katz, every one of those million pages inherits the *full* weight of `amazon.com` in the sum.

But that million pages should not all become important: each is *only one of a million* pages `amazon.com` chose to point at.

Fix (Brin & Page, 1998). When passing a node's score to its out-neighbors, *divide by its out-degree* so the total mass leaving the node is conserved.

PageRank = Katz centrality *normalized by out-degree.*

Equivalently: stationary distribution of a random walk on G with teleportation.

PageRank

Definition. Let G be a directed graph; let d_j^{out} be the out-degree of node j, and let $\alpha \in (0,1)$ be a *damping factor* (typically 0.85). The PageRank scores satisfy

$$x_i \;=\; \alpha \sum_{j:\, j \to i} \frac{x_j}{d_j^{\text{out}}} \;+\; (1-\alpha).$$

In matrix form: let $P = D_{\text{out}}^{-1} A$ be the *row-stochastic* transition matrix of the random walk (P_{ij} = probability of stepping from i to j). Then

$$x \;=\; \alpha\, P^{\top} x \;+\; (1-\alpha)\,\mathbf{1} \quad\Longleftrightarrow\quad x \;=\; (1-\alpha)\,(I - \alpha\, P^{\top})^{-1}\,\mathbf{1}.$$

(The transpose appears for the same reason as in Katz: the score of i aggregates over i's in-neighbors.)

Random-walk interpretation. A web surfer at node j does one of two things at each step:

- with probability α, follows a uniformly random out-link from j;
- with probability $1-\alpha$, *teleports* to a uniformly random page.

x_i is the long-run fraction of time the surfer spends at i.

Why teleportation? Without it, the walk gets stuck in dangling nodes / sink components. Teleportation makes the chain irreducible and aperiodic, so a unique stationary distribution exists.

3 Importance-based features (centrality measures)

3.5 Closeness centrality

Why closeness centrality?

A different intuition: instead of looking at a node's neighbors' scores, look at **shortest paths** in the network.

Let d_{ij} = shortest distance in G between nodes i and j (so $d_{ii} = 0$). The *mean distance* from node i is

$$\ell_i = \frac{1}{n} \sum_{j=1}^{n} d_{ij}.$$

A node with a small ℓ_i is influential: its opinions / news / rumors can spread quickly through the community.

But we want centrality scores to be *larger* for influential nodes, not smaller. So take the reciprocal $\Rightarrow$ closeness.

(*Aside.* Some authors average over $j \neq i$, dividing by $n - 1$. Either choice is a positive scaling that does not change rankings; we follow Newman's choice of n.)

Closeness centrality

Definition. Let $G = (V, E)$, $V = [n]$. The closeness centrality of node i is

$$C_i = \frac{1}{\ell_i} = \frac{n}{\sum_{j=1}^{n} d_{ij}}.$$

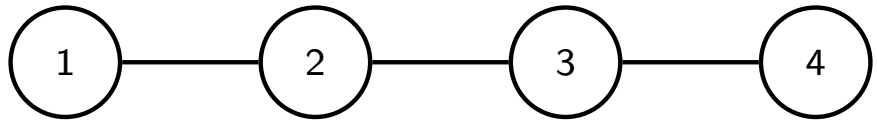
Caveat. If G is disconnected, $\ell_i = \infty$ and $C_i = 0$ for every node in a non-trivial component.

A common workaround restricts the sum to nodes in i's connected component (Newman 2018, p. 173).

Reading the formula. High closeness means: from i, every other node is *close* on average. Information broadcast from i reaches everyone in few hops.

Example: closeness centrality on P_4

Path on four vertices, $V = \{1, 2, 3, 4\}$.



Pairwise distance matrix:

$$(d_{ij}) = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 3 & 2 & 1 & 0 \end{pmatrix}.$$

Apply $C_i = n / \sum_j d_{ij}$ with $n = 4$:

i	$\sum_j d_{ij}$	$\ell_i = \frac{1}{n}\sum_{j=1}^{n} d_{ij}$	$C_i = 1/\ell_i$
1	6	3/2	$2/3 \approx 0.667$
2	4	1	1
3	4	1	1
4	6	3/2	$2/3 \approx 0.667$

The two interior nodes are tied for highest closeness; broadcast from 2 or 3 reaches every other node in ≤ 2 hops, while from an endpoint it takes 3.

3 Importance-based features (centrality measures)

3.6 Betweenness centrality

Why betweenness centrality?

A third intuition: a node is influential if it is a **gatekeeper** that information must pass through.

Idealized model. Each pair of nodes exchanges one message, along the shortest path between them. (In reality, traffic is uneven and messages do not always take shortest paths — but bear with this idealization.)

A node sitting on many such shortest paths sees a lot of traffic. It can:

- monitor or access the information that flows through it;
- charge a toll on every message it forwards;
- disrupt the flow by going offline.

$\Rightarrow$ Count how many shortest s–t paths pass through i, summed over all source–destination pairs.

Betweenness centrality

Definition. Let $G = (V, E)$, $V = [n]$, and assume there is a unique shortest path between any pair of nodes (for simplicity). Let

$$n_{st}^{i} = \begin{cases} 1, & \text{if node } i \text{ lies on the shortest } s\text{–}t \text{ path,} \\ 0, & \text{otherwise.} \end{cases}$$

The betweenness centrality of node i is

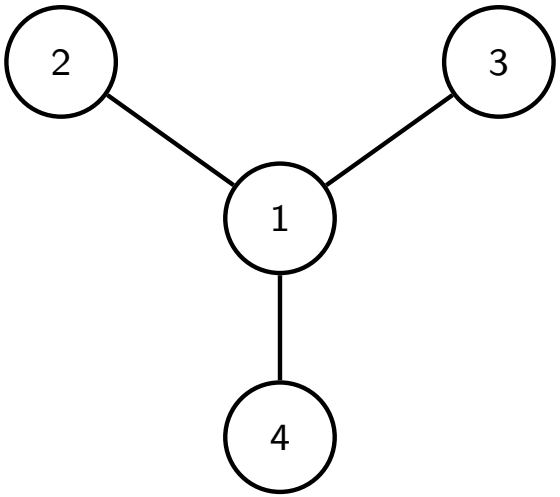
$$\boxed{x_i = \sum_{s=1}^{n} \sum_{t=1}^{n} n_{st}^{i}.}$$

Remarks.

- For undirected graphs one could divide by 2, but the formula above is convenient because it works for both cases. Scaling does not change rankings.
- If multiple shortest s–t paths exist, normalize the summand by g_{st}, the total number of such paths (i.e., assume the message picks one uniformly at random).

Example: betweenness centrality on the star $K_{1,3}$

Center 1, leaves $2, 3, 4$, edges $\{1,2\}, \{1,3\}, \{1,4\}$.



Sum is over all $n^2 = 16$ ordered pairs (s,t); $n_{st}^i = 1$ iff i lies on the (unique) shortest s–t path. Endpoints count as lying on their own paths.

Computing x_1 (the center):

- 1 is an *endpoint* of the path: pairs $(1,k)$ and $(k,1)$ for $k \in V$, contributing $4 + 4 - 1 = 7$ (removing the double-counted $(1,1)$).
- 1 is an *interior* vertex: every shortest path between two distinct leaves passes through 1, giving the $3 \cdot 2 = 6$ ordered pairs among $\{2,3,4\}$.

$\Rightarrow x_1 = 7 + 6 = 13$.

Computing x_2 (a leaf): 2 is endpoint in 7 pairs, interior in 0 pairs (it is a leaf).
$\Rightarrow x_2 = 7$. By symmetry $x_3 = x_4 = 7$.

3 Importance-based features (centrality measures)

3.7 Importance features in practice: Becchetti et al., web-spam

Why these features matter: web-spam detection

Becchetti et al., "Link analysis for Web spam detection" (2008).

Problem. *Web spam*: pages whose ranking is artificially inflated by deception, typically via dense *link farms* of fake pages all pointing at a target. A multi-billion-dollar problem for search-engine quality in the 2000s.
Task: binary node classification on the web graph.

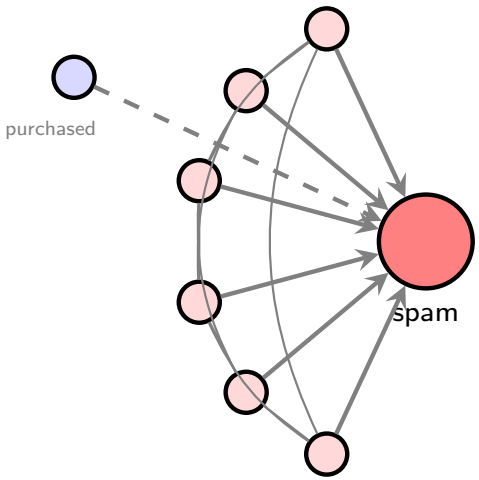
Graph. Two large `.uk` crawls:
`UK-2002`: 18.5M pages, 298M links, 98,542 hosts;
`UK-2006`: 77.9M pages, 3.0B links, hand-labeled subsets of $\sim$ 5K hosts each.

Features (163-dim vector per host, link-based only, no page content). Built directly from quantities of this section:

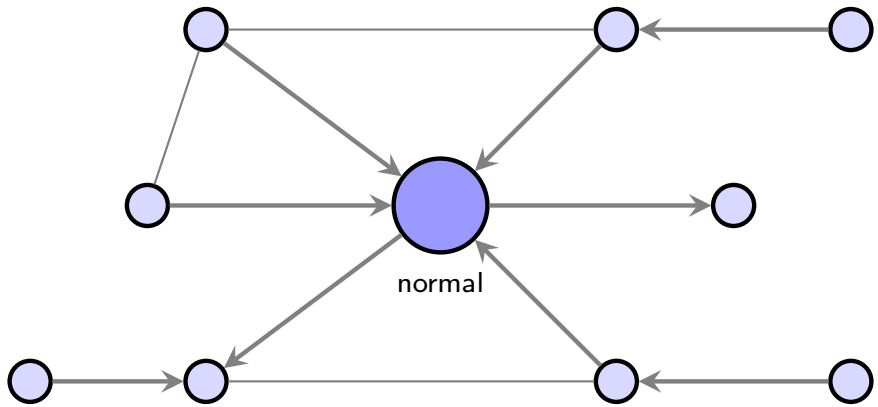
- *Degree-based*: log in-degree, log out-degree, edge-reciprocity, neighbor-degree assortativity.
- *Centrality-based*: PageRank, TrustRank, Truncated PageRank.
- *Neighborhood-counting*: estimated #supporters at distances $d = 1, 2, 3, 4$ (a directed analog of $|N_d(v)|$).
- *Shape descriptors*: ratios of the above across distances and pages.

Fed into a $C4.5$ decision tree with bagging — generic, off-the-shelf classifier.

Web-spam detection: the asymmetry of link structure



dense link farm



organically connected web

Headline (10-fold CV, F_1). 0.871 on UK-2002, **0.631** on UK-2006 — *comparable* to the best content-based classifiers of the era and *orthogonal* to them (combining improves further).

Three lessons.

(1) Hand-crafted node features carry serious signal at scale.
(2) Different feature *families* (degree, centrality, distance-graded counts) are not redundant — they combine multiplicatively, not additively.
(3) These features encode *hypotheses about structure* that GNNs will later *learn* (e.g., Truncated PageRank is a hand-coded message-passing scheme).

(Figure adapted from Becchetti et al., Fig. 2.)

4 Structure-based features

4 Structure-based features

4.1 Clustering coefficient

Transitivity and the clustering coefficient

In many social networks: if $u \sim v$ and $v \sim w$, it is more likely (though not guaranteed) that $u \sim w$.

Terminology.

- $u-v-w$ with $u \sim v$ and $v \sim w$ is a 2-path (a.k.a. *open triad* if $u \not\sim w$, *connected triple*).
- If additionally $u \sim w$, transitivity holds and the 2-path is closed (a.k.a. *closed triad*, i.e. a triangle).

Definition (global clustering coefficient). For an undirected graph $G = (V, E)$,

$$C = \frac{\#\,\text{closed 2-paths}}{\#\,\text{2-paths}}.$$

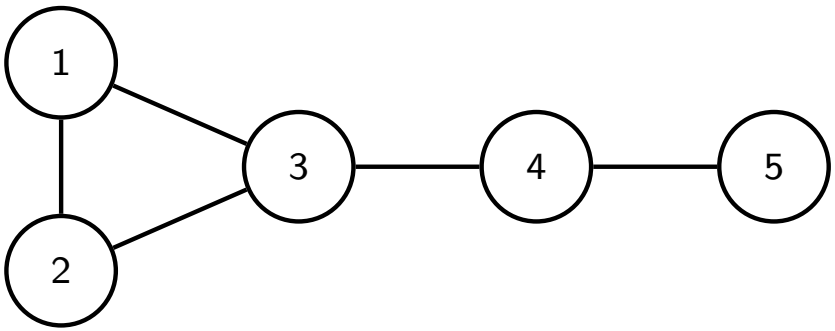
Equivalent forms (counting ordered 2-paths, or connected triples aka $K_{1,2}$):

$$C = \frac{6 \times \#\,\text{triangles}}{\#\,\text{(ordered) 2-paths}} = \frac{3 \times \#\,\text{triangles}}{\#\,\text{connected triples}}.$$

$C \in [0, 1]$. $C = 0 \Leftrightarrow$ no triangles. $C = 1 \Leftrightarrow$ every component is a clique.

Empirically: real social networks have $C \sim 0.2$; random graphs of the same size have $C \sim 10^{-4}$. People do not pick friends uniformly at random.

Example: global clustering coefficient on a triangle-with-tail



Edges: $\{1,2\},\{1,3\},\{2,3\},\{3,4\},\{4,5\}$.
Degrees: $d_1 = d_2 = 2,\ d_3 = 3,\ d_4 = 2,\ d_5 = 1$.

Triangles. The number of triangles is $\mathrm{T} = 1$ (only $\{1,2,3\}$ is pairwise adjacent).

(Ordered) 2-paths. Centred at v there are $d_v(d_v - 1)$ ordered 2-paths.

$$\sum_v d_v(d_v - 1) \;=\; 2 + 2 + 6 + 2 + 0 \;=\; 12.$$

Closed (ordered) 2-paths. Each triangle on $\{a, b, c\}$ gives the six closed 2-paths $abc, acb, bac, bca, cab, cba$, so $6\,\mathrm{T} = 6$.

$$C \;=\; \frac{6}{12} \;=\; \boxed{\frac{1}{2}}.$$

Sanity check via connected triples: $K_{1,2}$-count $= 6$, so $C = 3 \cdot \mathrm{T}/6 = 1/2$. ✓

Local clustering coefficient: a per-node feature

Motivation (Watts & Strogatz). The *global* C is one number for the whole graph — wrong granularity for node classification. Two nodes with similar degree and similar eigenvector centrality may play different *structural roles*: one inside a tight clique, the other a star-like hub with unconnected neighbors. Degree and eigenvector centrality cannot tell them apart; *the proportion of edges among a node's neighbors* can.

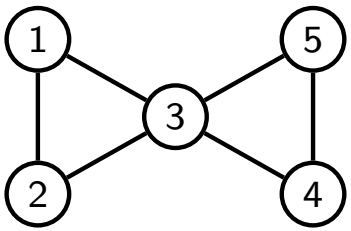
Definition (local clustering coefficient). For node u with $d_u \geq 2$,

$$C_u = \frac{\left|\{\{v_1, v_2\} \in E : v_1, v_2 \in N(u)\}\right|}{\binom{d_u}{2}}.$$

Equivalently, $C_u = \dfrac{\#\,\text{triangles through } u}{\#\,\text{unordered pairs of neighbors of } u}$, the *edge density of the induced subgraph* $G[N(u)]$.

Convention. If $d_u \in \{0, 1\}$ the denominator is zero and C_u is set to 0.

Example: local clustering coefficient on the bowtie



Degrees: $d_1 = d_2 = d_4 = d_5 = 2,\ d_3 = 4$.
For each u: read off $N(u)$, count edges of G both of whose endpoints lie in $N(u)$, divide by $\binom{d_u}{2}$.

u	$N(u)$	$\binom{d_u}{2}$	$\#\{\{v_1, v_2\} \in E : v_1, v_2 \in N(u)\}$	C_u
1	$\{2, 3\}$	1	1 (edge $\{2, 3\}$)	1
2	$\{1, 3\}$	1	1 (edge $\{1, 3\}$)	1
3	$\{1, 2, 4, 5\}$	6	2 (edges $\{1, 2\}, \{4, 5\}$)	$1/3$
4	$\{3, 5\}$	1	1 (edge $\{3, 5\}$)	1
5	$\{3, 4\}$	1	1 (edge $\{3, 4\}$)	1

Every non-hub node has $C_u = 1$ ...
Every non-hub node has $C_u = 1$ (its two neighbors are themselves adjacent). The hub 3 has $C_3 = 1/3$ — the four *cross-triangle* pairs $\{1, 4\}, \{1, 5\}, \{2, 4\}, \{2, 5\}$ are all non-edges.
Punchline. Degree gave $d_3 = 4$ as the most central, all others tied. *Clustering coefficient does the opposite*: 3 is the *least* clustered — correctly identifying it as a structural **bridge**, not a community member.

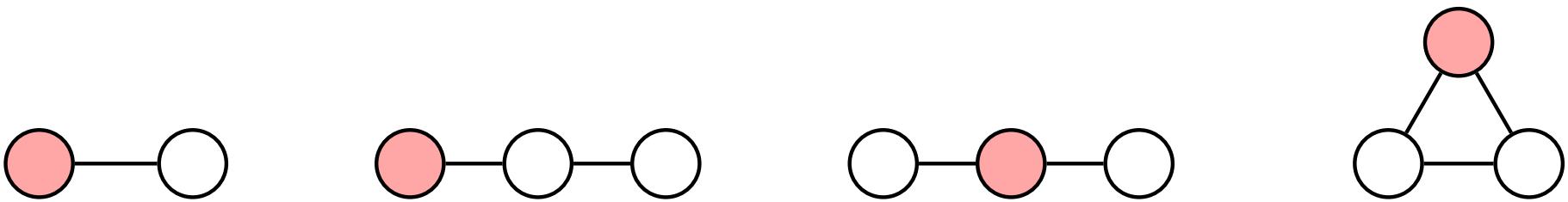
4 Structure-based features

4.2 Why a richer fingerprint? Towards the graphlet count vector

From clustering coefficient to a richer node fingerprint

The clustering coefficient C_u summarizes *one* facet of the local structure around u: the density of triangles through u.

But u may participate in many other small substructures that C_u is blind to:



edge K_2 | P_3 endpoint | P_3 middle | triangle K_3

u may be the *middle* of an induced P_3 or one of its *endpoints*; the *hub* of a star $K_{1,3}$ or one of its *leaves*; a vertex of an induced C_4; ...

Idea. Count how many small substructures of *each type* pass through u, and assemble the counts into a single integer-valued vector. One coordinate per (graphlet, anchor position) pair.

The result is the **Graphlet Degree Vector** (GDV) of Pržulj (2007). It strictly refines the clustering coefficient (C_u is one of its coordinates) and discriminates nodes that no scalar centrality measure can tell apart.

Why GDV (1/2): cancer-gene discovery from network shape alone

Milenković & Pržulj, "Uncovering biological network function via graphlet degree signatures" (2008).

The question. Why should a node's structural fingerprint — a vector of small-substructure counts — predict anything biologically meaningful?

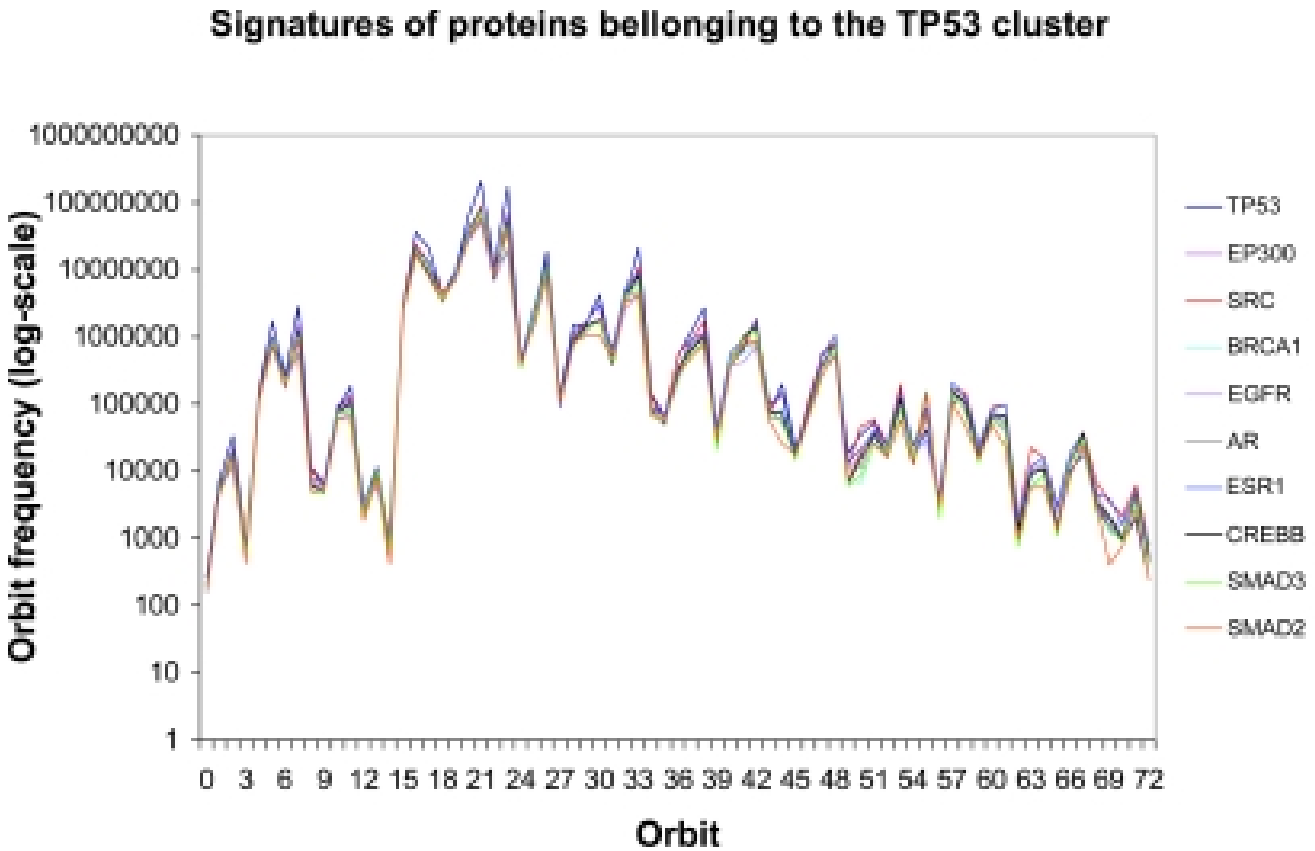
The setup. A protein–protein interaction (PPI) network on $\sim$ 10,488 human proteins ($\sim$ 60,000 interactions); each protein is *one vertex*. The 73-coordinate GDV is computed for every protein. They cluster proteins by GDV signature, then look at the cluster around **TP53**, the most-studied human cancer gene.

The result. The 10-protein TP53 cluster includes **BRCA1, EGFR, EP300, SRC, AR** — six of the most clinically important cancer genes in the human genome. Eight of the ten are known disease genes; six are known cancer genes.

Recovered using nothing but the local shape of the network around each protein.

A feature defined purely by graph topology, with no biological semantics built in, lining up with biology that took decades of laboratory work to map.

Why GDV (1/2): the TP53 cluster's GDV signatures



Horizontal axis: 73 automorphism orbits of graphlets on 2–5 nodes. Vertical (log) axis: the count for that orbit. All ten signatures lie within signature similarity ≥ 0.95 of TP53 — they trace nearly the same curve across the 73 coordinates.

(Figure: Milenković & Pržulj, 2008, Fig. 6, CC-BY 3.0; PMC2623288.)

Why GDV (2/2): GDV beats the 1-WL ceiling on ZINC

Bouritsas, Frasca, Zafeiriou & Bronstein, "Improving graph neural network expressivity via subgraph isomorphism counting" (2023).

The setup. Inject GDV-style orbit counts as node features into a standard message-passing GNN ⇒ *Graph Substructure Networks* (GSN).

The theory.

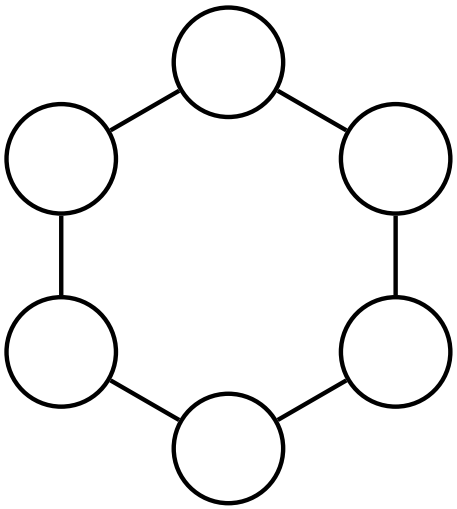
- Standard MPNNs are upper-bounded by the 1-WL test (Xu et al. 2019; cf. Session 3).
- 1-WL-bounded GNNs provably *cannot* count triangles or larger substructures (Chen, Villar, Chen & Bruna 2020).
- GSN can. So GSN is strictly more expressive than every standard MPNN.

The empirics (ZINC-12k, constrained-solubility regression).

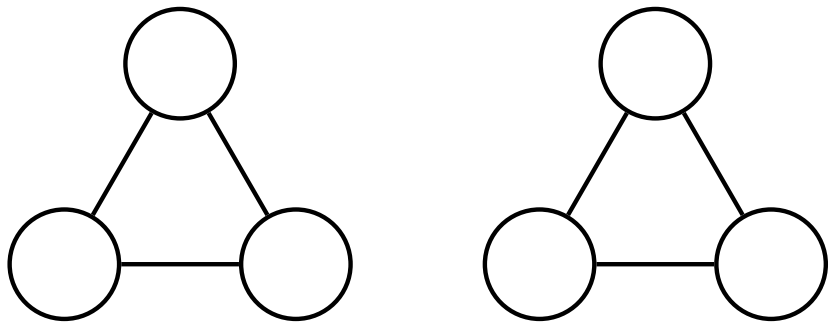
Model	**Test MAE**
Vanilla GIN (1-WL bounded)	~ 0.25
GSN (with orbit counts)	$\mathbf{0.10 - 0.14}$

On a benchmark where 0.01 improvements routinely count as state-of-the-art, this is a meaningful gap. Similar gains on OGB-MolHIV and several TUD datasets.

Why GDV (2/2): one tiny pair where 1-WL fails, GDV succeeds



C_6



$C_3 \sqcup C_3$

Both graphs have 6 vertices, 6 edges, every vertex of degree 2. They are 2-regular and locally identical to 1-WL — it hashes them to identical color histograms. So no standard MPNN can tell them apart.

But the triangle count $\gamma_3(u)$ separates them instantly:

- Every vertex of C_6 lies in 0 triangles.
- Every vertex of $C_3 \sqcup C_3$ lies in 1 triangle.

4 Structure-based features

4.3 Graphlet count vector: definition and examples

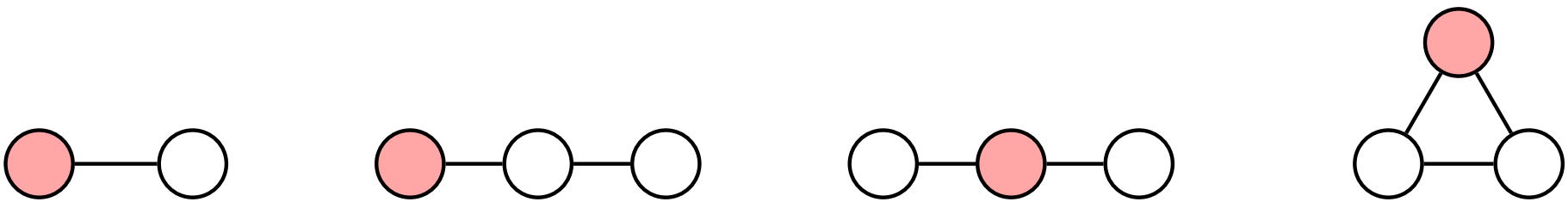
Rooted graphlets: setup

Let H be a connected simple graph on k vertices.

- $\mathrm{Aut}(H)$ = group of edge-preserving bijections $V(H) \to V(H)$.
- Its action partitions $V(H)$ into *automorphism orbits*: u, v in the same orbit iff some automorphism maps $u \mapsto v$.

Definition. A rooted graphlet is a pair $(H, \mathcal{O})$ with $\mathcal{O} \subseteq V(H)$ an automorphism orbit. We pick a representative $v^\star \in \mathcal{O}$ as the anchor.

Restricting to graphlets on ≤ 3 vertices, there are exactly *four* rooted graphlets:



γ_0: K_2 γ_1: P_3 endpoint γ_2: P_3 middle γ_3: K_3

P_3 has *two* orbits (endpoint and middle), since $\mathrm{Aut}(P_3) = \mathbb{Z}/2$ swaps endpoints. K_2 and K_3 each have one orbit (S_2, S_3 act transitively).

Graphlet degree vector (GDV)

Definition. For node $u \in V(G)$ and rooted graphlet index $i \in \{0, 1, 2, 3\}$, let

$$\gamma_i(u) \;=\; \Big|\Big\{\, S \subseteq V(G) :\; u \in S,\;\; \exists \text{ iso. } \varphi : G[S] \xrightarrow{\sim} H_i \text{ with } \varphi(u) \in \mathcal{O}_i \,\Big\}\Big|.$$

(Whether we write "$\exists\,\varphi$" or "$\forall\,\varphi$" makes no difference: any two isomorphisms $G[S] \to H_i$ differ by an automorphism of H_i, which permutes vertices within each orbit.)

The graphlet degree vector of u at depth ≤ 3 is

$$\boxed{\gamma(u) \;=\; \big(\gamma_0(u),\, \gamma_1(u),\, \gamma_2(u),\, \gamma_3(u)\big) \;\in\; \mathbb{Z}^4_{\geq 0}.}$$

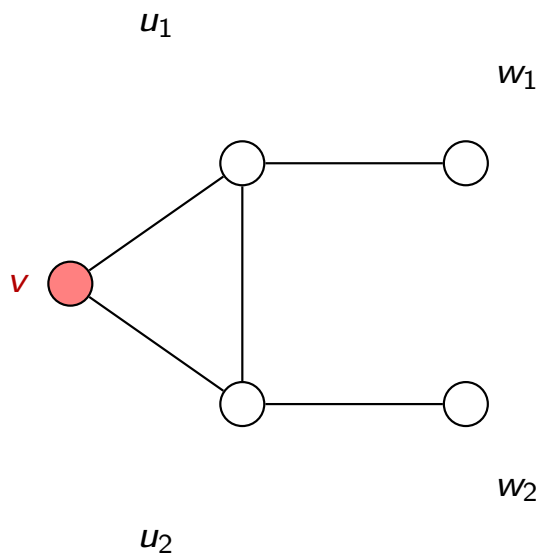
Extending to all rooted graphlets on at most:

- 4 vertices $\Rightarrow \gamma(u) \in \mathbb{Z}^{15}_{\geq 0}$
- 5 vertices $\Rightarrow \gamma(u) \in \mathbb{Z}^{73}_{\geq 0}$ (the standard GDV in bioinformatics)

Range of the GDV.

- Reaches exactly 4 hops (for the depth-5 version): any vertex contributing to $\gamma(u)$ lies within distance ≤ 4 of u.
- Strictly refines C_u: $C_u = \gamma_3(u) / \binom{\gamma_0(u)}{2}$.

Example 1: GDV of the apex of a triangle-with-pendants (1/2)



$V = \{v, u_1, u_2, w_1, w_2\}$,
$E = \{vu_1, vu_2, u_1u_2, u_1w_1, u_2w_2\}$.

v is the apex of the triangle $\{v, u_1, u_2\}$; u_1, u_2 each carry a pendant.

Goal: compute $\gamma(v)$ at depth ≤ 3.

$\gamma_0(v)$ **(K_2, edge).**

- Count edges incident to v: $\gamma_0(v) = \deg(v) = 2$.
- Contributing subsets: $\{v, u_1\}$ and $\{v, u_2\}$.

$\gamma_1(v)$ **(P_3, endpoint).** Find an induced P_3 with v at an endpoint: pick a neighbor u of v, then a vertex $x \in N(u) \setminus (\{v\} \cup N(v))$.

- Through u_1: $N(u_1) = \{v, u_2, w_1\} \Rightarrow x = w_1$.
- Through u_2: by symmetry, $x = w_2$.
- Total: $\gamma_1(v) = 2$.

Example 1: GDV of the apex of a triangle-with-pendants (2/2)

(Same graph as previous slide; goal is to finish computing $\gamma(v)$.)

$\gamma_2(v)$ **(P_3, middle).** Find an induced P_3 with v in the middle: need two *non-adjacent* neighbors of v.

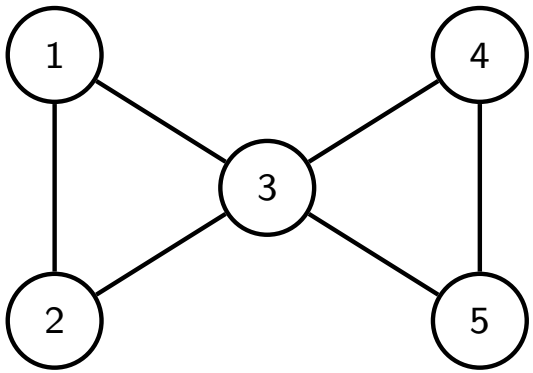
- Only candidate pair: (u_1, u_2).
- But $u_1u_2 \in E$, so $G[\{v, u_1, u_2\}]$ is a triangle, not a P_3.
- No valid configuration: $\gamma_2(v) = 0$.

$\gamma_3(v)$ **(K_3, triangle).** Find a triangle through v: need two *adjacent* neighbors of v.

- Only candidate pair: (u_1, u_2), with $u_1u_2 \in E$. ✓
- One triangle: $\{v, u_1, u_2\}$.
- Total: $\gamma_3(v) = 1$.

$$\boxed{\gamma(v) = (2, 2, 0, 1).}$$

Example 2: GDV of the bowtie graph (per-node)



Symmetry. Aut has order 8: vertex 3 is fixed, $\{1,2,4,5\}$ form a single orbit. $\Rightarrow$ expect $\gamma(1) = \gamma(2) = \gamma(4) = \gamma(5)$, distinct from $\gamma(3)$.

$\gamma(3)$: $N(3) = \{1,2,4,5\}$.

- $\gamma_0 = \deg(3) = 4$.
- $\gamma_1 = 0$: $N(3) = V \setminus \{3\}$, so no outside vertex exists.
- $\gamma_2 = 4$: of the $\binom{4}{2} = 6$ pairs in $N(3)$, $\{1,2\}$ and $\{4,5\}$ are edges; the other four give induced P_3's.
- $\gamma_3 = 2$: triangles $\{1,2,3\}, \{3,4,5\}$.

$\Rightarrow \gamma(3) = (4,\ 0,\ 4,\ 2)$.

$\gamma(1)$: $N(1) = \{2,3\}$.

- $\gamma_0 = 2$.
- $\gamma_1 = 2$: paths $1-3-4$ and $1-3-5$.
- $\gamma_2 = 0$: only candidate pair $\{2,3\} \in E$.
- $\gamma_3 = 1$: triangle $\{1,2,3\}$.

$\Rightarrow \gamma(1) = (2,\ 2,\ 0,\ 1)$.

By symmetry, $\gamma(2) = \gamma(4) = \gamma(5) = \gamma(1)$.

Example 2 (cont.): GDV table for the bowtie

Node u	$\gamma_0(u)$	$\gamma_1(u)$	$\gamma_2(u)$	$\gamma_3(u)$	$C_u = \gamma_3 / \binom{\gamma_0}{2}$
1	2	2	0	1	1
2	2	2	0	1	1
3	4	0	4	2	1/3
4	2	2	0	1	1
5	2	2	0	1	1

Three observations.

1. GDV recovers the automorphism orbits $\{3\}$ and $\{1, 2, 4, 5\}$ from purely local counts.
2. Node 3 is uniquely identified by the pattern $\gamma_1 = 0,\ \gamma_2 \gg 0$: it never sits at the *periphery* of an induced path, but it is the joint of every induced path bridging the two triangles — the *articulation vertex*.
3. C_u is recovered from a *single* ratio $\gamma_3 / \binom{\gamma_0}{2}$ of GDV coordinates, leaving the other coordinates with information not in C_u. The GDV *strictly refines* the clustering coefficient.

Among hand-engineered node features in the pre-GNN toolkit, the GDV is the most discriminative.

5 Preliminaries: the 1-WL algorithm

The Weisfeiler–Leman algorithm: setup

The graph-level features and kernels we develop next sit on top of the **Weisfeiler–Leman** (WL) algorithm.

WL was introduced by Weisfeiler and Leman in 1968 as a polynomial-time *heuristic* for the graph isomorphism problem. It refines a node coloring by iteratively hashing each node's current label together with the multiset of its neighbors' labels. Also called *color refinement* or *naive vertex classification*.

1-WL = 1-dimensional WL: refines colors of *individual nodes* (1-tuples of vertices). For $k \geq 2$, k-WL refines colors of k-tuples and is strictly more expressive.

Notation (per iteration i).

- $\ell_i(v)$ = label of node v.
- $M_i(v) = \{\!\{\ell_{i-1}(u) : u \in N(v)\}\!\}$ = multiset of neighbor labels.
- $s_i(v) = \ell_{i-1}(v) \,\|\, \mathrm{sort}(M_i(v))$ = sorted-and-prefixed string.
- $\ell_i(v) = f(s_i(v))$ for a single *injective* hash $f : \Sigma^* \to \Sigma$ (shared across both graphs).

The 1-WL isomorphism test (pseudocode)

Algorithm 1-WL isomorphism test, $\mathrm{WL}(G, G', n)$

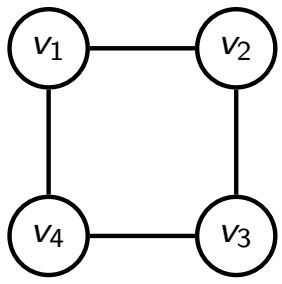
Input: Two undirected graphs G, G' on n vertices each; injective hash f
Output: "Not isomorphic" (certifies $G \not\cong G'$) or "1-WL indistinguishable"

$\ell_0(v) \leftarrow \deg(v)$ for all $v \in V(G) \cup V(G')$ ▷ initialize
for $i = 1, 2, \ldots, n$ **do**
  for each $v \in V(G) \cup V(G')$ **do**
    $M_i(v) \leftarrow \{\!\{\, \ell_{i-1}(u) : u \in N(v) \,\}\!\}$ ▷ aggregate neighbors
    $s_i(v) \leftarrow \ell_{i-1}(v) \,\|\, \mathrm{sort}\big(M_i(v)\big)$ ▷ sort + concat
    $\ell_i(v) \leftarrow f\big(s_i(v)\big)$ ▷ compress to new label
  end for
  if $\{\!\{\ell_i(v) : v \in V(G)\}\!\} \neq \{\!\{\ell_i(u) : u \in V(G')\}\!\}$ **then**
    return "Not isomorphic" ▷ certificate of $G \not\cong G'$
  end if
end for
return "1-WL indistinguishable"

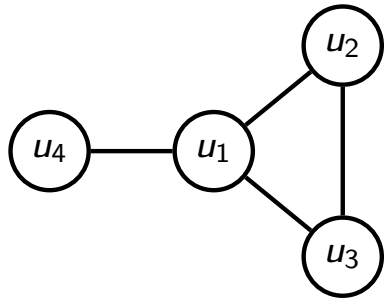
Two facts about the algorithm.

- **Why n iterations suffice.** The partition of $V(G) \cup V(G')$ induced by ℓ_i is a *refinement* of the partition induced by ℓ_{i-1} (the hash is injective on the prefix). Refinement can happen at most $n - 1$ times before the partition stabilizes.
- **Sound but not complete.** If WL outputs "Not isomorphic", then $G \not\cong G'$ (isomorphic graphs always produce identical label multisets). The converse fails: there are non-isomorphic graphs that 1-WL cannot distinguish.

1-WL example 1: different degree sequences (C_4 vs. K_3+pendant) (1/2)



$G = C_4$



$G' = K_3 + \textit{pendant}$

Both graphs have 4 vertices and 4 edges, but their degree sequences differ:

$$\deg(G) = (2,2,2,2) \qquad \text{vs.} \qquad \deg(G') = (3,2,2,1).$$

At iteration 0 the multisets of labels are

$$\{\{2,2,2,2\}\} \neq \{\{3,2,2,1\}\},$$

so the algorithm could halt immediately and certify $G \ncong G'$.

On the next slide, we run one full iteration anyway — to see how the hash machinery refines labels even when refinement wasn't needed.

1-WL example 1: different degree sequences (C_4 vs. K_3+pendant) (2/2)

Run iteration 1 with hash $f(13) = A,\ f(222) = B,\ f(223) = C,\ f(3122) = D$:

v	v_1	v_2	v_3	v_4	u_1	u_2	u_3	u_4
$\ell_0(v)$	2	2	2	2	3	2	2	1
$M_1(v)$	$\{2,2\}$	$\{2,2\}$	$\{2,2\}$	$\{2,2\}$	$\{2,2,1\}$	$\{3,2\}$	$\{3,2\}$	$\{3\}$
$s_1(v)$	222	222	222	222	3122	223	223	13
$\ell_1(v)$	B	B	B	B	D	C	C	A

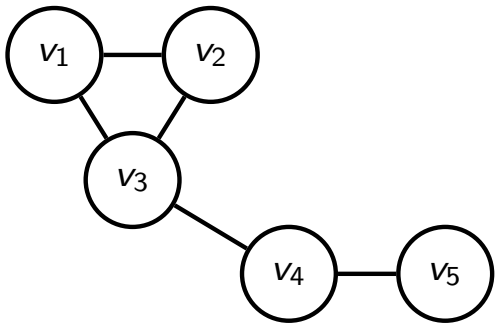
Comparing the iteration-1 label multisets:

$$\{\{B,B,B,B\}\} \neq \{\{D,C,C,A\}\} \quad \Rightarrow \quad \text{“Not isomorphic”}.\ \checkmark$$

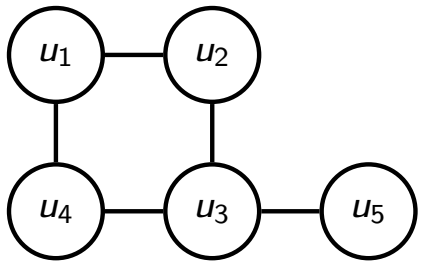
What this illustrates. 1-WL refines a node's label by combining its current label with the sorted multiset of neighbor labels. Here, distinct degrees were enough on their own — but the iteration-1 rows show how every vertex of G now carries the same label B, while every vertex of G' carries a different label, exposing the structural mismatch even more starkly.

1-WL example 2: same degree sequence (1/2)

Both graphs have degree sequence $(1, 2, 2, 2, 3)$:



G: triangle + pendant path



G': 4-cycle + pendant

G contains a triangle (not bipartite); G' has only the 4-cycle (bipartite). So $G \ncong G'$ — but degree sequences alone cannot tell them apart.

Iteration 0: same degree multiset $\{\{1, 2, 2, 2, 3\}\}$ on both sides $\Rightarrow$ no conclusion. We need at least one round of refinement.

1-WL example 2: same degree sequence (2/2)

Run iteration 1 with shared hash
$f(213) = C,\ f(222) = D,\ f(223) = E,\ f(3122) = F,\ f(3222) = G, \ldots$:

v	v_1	v_2	v_3	v_4	v_5	u_1	u_2	u_3	u_4	u_5
ℓ_0	2	2	3	2	1	2	2	3	2	1
M_1	$\{2,3\}$	$\{2,3\}$	$\{2,2,2\}$	$\{1,3\}$	$\{2\}$	$\{2,2\}$	$\{2,3\}$	$\{1,2,2\}$	$\{2,3\}$	$\{3\}$
s_1	223	223	3222	213	12	222	223	3122	223	13
ℓ_1	E	E	G	C	A	D	E	F	E	B

Comparing the iteration-1 label multisets:

$$\{\{A, C, E, E, G\}\} \neq \{\{B, D, E, E, F\}\} \Rightarrow \text{“Not isomorphic”}. \checkmark$$

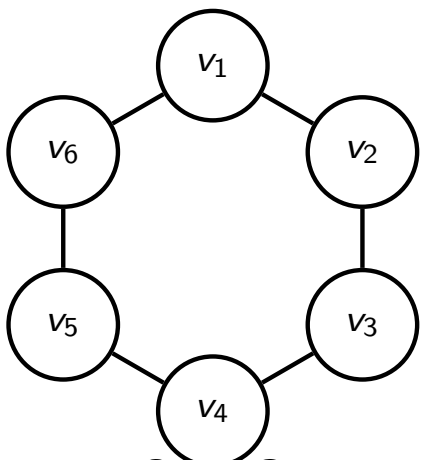
What did 1-WL detect? In G, the degree-3 vertex has all three neighbors of degree 2 ($s_1 = 3222$). In G', the degree-3 vertex has one degree-1 and two degree-2 neighbors ($s_1 = 3122$). One asymmetry is enough.

1-WL example 3: C_6 vs. $C_3 \sqcup C_3$ (1/2)

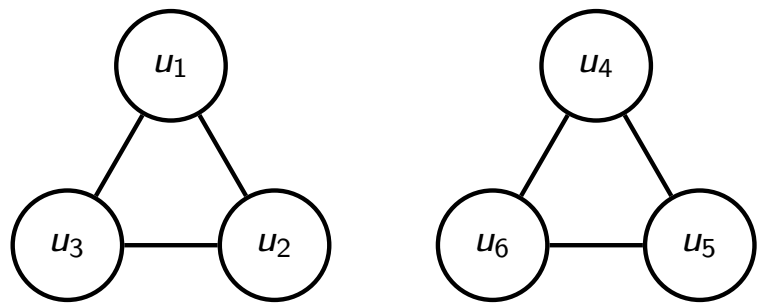
The canonical pair on which 1-WL fails:

$$G_1 = C_6 \text{ (connected, girth 6)}, \qquad G_2 = C_3 \sqcup C_3 \text{ (disconnected, girth 3)}.$$

Both have 6 vertices, 6 edges, every vertex of degree 2. Clearly $G_1 \ncong G_2$.



$G_1 = C_6$



$G_2 = C_3 \sqcup C_3$

Iteration 0. $\ell_0(v) = 2$ for every v in either graph (all vertices have degree 2). The label multisets are

$$\{\{2,2,2,2,2,2\}\} \;=\; \{\{2,2,2,2,2,2\}\},$$

identical on both sides — iteration 0 gives no information. On the next slide, we show by induction that no later iteration does either.

1-WL example 3: C_6 vs. $C_3 \sqcup C_3$ (2/2)

Inductive step. Suppose at iteration i every vertex of $G_1 \cup G_2$ shares a common label λ_i. Then for every v,

$$M_{i+1}(v) = \{\{\lambda_i, \lambda_i\}\}, \qquad s_{i+1}(v) = \lambda_i \,\|\, \lambda_i\lambda_i,$$

the same string everywhere. So $\ell_{i+1}(v) = \lambda_{i+1}$, again common to all vertices. Induction holds.

The label multisets agree at every iteration $\Rightarrow$ 1-WL never reaches the "Not isomorphic" branch, and outputs "1-WL indistinguishable".

Why does 1-WL fail here? Both graphs are 2-regular and *locally* indistinguishable: every r-hop neighborhood is a path of length $\min(r, \mathrm{girth}/2)$. There is no local asymmetry for the multiset-of-neighbor-labels rule to latch onto. The only difference — G_1 has one cycle of length 6, G_2 has two cycles of length 3 — is a *global* property invisible to local message passing.

Beyond 1-WL. Distinguishing $(C_6,\ C_3 \sqcup C_3)$ requires k-WL with $k \geq 2$, or features beyond 1-WL such as triangle counts (each vertex of G_2 lies in 1 triangle; each vertex of G_1 lies in 0). This is the same observation we made earlier when introducing the GDV.

6 Graph kernels

Graph-level features: from nodes to whole graphs

Many ML tasks are at the **graph level** — given an entire graph G, predict one label for it.

Examples.

- *Molecular property prediction.* Is this molecule toxic? soluble? biologically active?
- *Protein function prediction.* What does this protein do?
- *Social-network classification.* What kind of small community is this?

Three classical strategies.

1. **Bag of nodes.** Sum (or histogram) the per-node feature vectors developed earlier (importance- and structure-based).
2. **Weisfeiler–Leman kernel.** Run color refinement; histogram the labels at each iteration.
3. **Graphlet / path-based kernels.** Count occurrences of small subgraphs or short walks/paths.

The bag of nodes is the simplest. Its weakness: *structural myopia* — two graphs with identical multisets of per-node features get identical bag-of-nodes vectors, even when they differ dramatically in large-scale connectivity. The WL kernel was designed to fix this.

The Weisfeiler–Leman kernel

Idea. Run 1-WL color refinement on G. After each iteration $i = 0, 1, \ldots, K$, count the occurrences of each label.

Definition. The WL feature vector of G at depth K is the histogram

$$\Phi_{\mathrm{WL},K}(G)[\ell] \;=\; \big|\{v \in V : \ell_K(v) = \ell\}\big|, \quad \ell \in \Sigma_K,$$

concatenated across iterations $0, 1, \ldots, K$.

The WL kernel of depth K is

$$\boxed{k_{\mathrm{WL},K}(G, G') \;=\; \big\langle \Phi_{\mathrm{WL},K}(G),\, \Phi_{\mathrm{WL},K}(G') \big\rangle.}$$

Cross-graph bookkeeping. The hash f must assign the *same* fresh label to the same string $s_i(v)$ wherever it occurs across the dataset. Histogram indices are then aligned and the inner product is meaningful.

Cost. Linear in $|E|$ per iteration, on each graph independently.

WL kernel: bag-of-nodes connection (1/2)

Bag of nodes = depth-0 WL. If we initialize $\ell_0(v) = \deg(v)$, the depth-0 WL feature vector is just the degree histogram of G — the bag-of-nodes feature.

Each additional iteration mixes a node's label with its neighbors' labels:
depth 0: node degree; depth 1: neighbors' degrees; depth 2: 2-hop neighborhood; ...

Worked example: P_5 vs. P_3. Run 1-WL on each (init. from degrees, hash shared across the two graphs).

Labelings.

$$P_5 : \ell_0 = (1, 2, 2, 2, 1), \quad \ell_1 = (A, B, C, B, A).$$
$$P_3 : \ell_0 = (1, 2, 1), \quad \ell_1 = (A, D', A).$$

(Endpoints of P_3 get the same label A as endpoints of P_5 — they hash the same string "12". The center of P_3 hashes a new string "211", getting a fresh label D'.)

Histograms over the shared alphabet $\{1, 2\} \sqcup \{A, B, C, D'\}$, concatenated:

$$\Phi_{\mathrm{WL},1}(P_5) = (2, 3,\ 2, 2, 1, 0), \qquad \Phi_{\mathrm{WL},1}(P_3) = (2, 1,\ 2, 0, 0, 1).$$

(Each graph maps to one vector — the input-to-output transformation.)

WL kernel: depth helps, and forward connection (2/2)

From the previous slide:

$$\Phi_{\mathrm{WL},1}(P_5) = (2, 3,\ 2, 2, 1, 0), \qquad \Phi_{\mathrm{WL},1}(P_3) = (2, 1,\ 2, 0, 0, 1).$$

Inner product gives the kernel.

$$k_{\mathrm{WL},1}(P_5, P_3) \ = \ 4 + 3 + 4 + 0 + 0 + 0 \ = \ 11, \qquad \text{cosine} \approx 0.74.$$

Going to depth 2 drops the cosine to ≈ 0.51 — deeper WL sees more, sees more difference.

Why depth helps. At depth 0 the kernel only compares degree distributions. At depth 1 it compares the multiset of (node, neighbors-of-node) patterns. At depth K it compares the multiset of K-hop rooted subtrees. Two graphs that look alike at low depths can become arbitrarily different at high depths, provided 1-WL can distinguish them at all (recall C_6 vs. $C_3 \sqcup C_3$ on slide 56 — 1-WL never separates them at any depth).

Forward connection. The WL iteration — aggregate neighbor labels, hash to a new label — is structurally identical to a message-passing GNN layer (Session 6). Replacing the hash by a differentiable network gives the GIN architecture (Xu et al. 2019), which is exactly as expressive as 1-WL. The WL kernel is the non-parametric ancestor of the MPNN.

7 Neighborhood overlap detection (link prediction)

Why a different feature for link prediction?

The features so far have been per-node ($\phi(v) \in \mathbb{R}^d$) or per-graph ($\Phi(G) \in \mathbb{R}^d$).

But **link prediction** (a.k.a. relation prediction) asks: *will an edge* $\{u, v\}$ *form?* The output is indexed by an *unordered pair* of nodes.

Definition. A similarity matrix is $S \in \mathbb{R}^{n \times n}$ with $S[u, v] =$ score for the $\{u, v\}$ relationship. A threshold-based link-prediction rule declares $\{u, v\}$ to be an edge whenever $S[u, v] > \tau$.

Evaluation. Compute S on a subsampled training graph $G_{\text{train}} = (V, E_{\text{train}})$ with $E_{\text{train}} \subsetneq E$; evaluate predictions on the held-out edges $E \setminus E_{\text{train}}$.

Why per-node features alone don't suffice. You could define $S[u, v] = \langle \phi(u), \phi(v) \rangle$ or $S[u, v] = -\|\phi(u) - \phi(v)\|$, but these ad-hoc combinations do not exploit the *joint local neighborhood structure* that is most predictive of a new edge.

Familiar example. Facebook's "People you may know" uses $|N(u) \cap N(v)|$. The signal comes from the *intersection of neighborhoods*, not from any feature of u or v alone.

7 Neighborhood overlap detection (link prediction)

7.1 Common neighbors and a threshold example

Common-neighbors index

Definition. The common-neighbors index is

$$S_{\mathrm{CN}}[u, v] \;=\; |N(u) \cap N(v)|.$$

The simplest of the local overlap measures: just count mutual friends.

Threshold-based rule. Recommend $\{u, v\}$ as a likely future edge whenever $S_{\mathrm{CN}}[u, v] \geq \tau$.

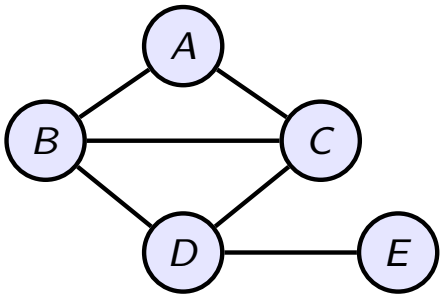
Why it works. Two users with many mutual friends are more likely to know each other (or soon will) than two users with none.

Where it's used.

- Facebook "People you may know"
- LinkedIn "People you may know"
- Twitter (early) "Who to follow"
- Friend recommendations on most consumer social networks

The signal is so strong that real-world deployments occasionally produce *uncomfortably* accurate suggestions (a therapist's other patients, an estranged relative) — mutual friends leak information no individual user has explicitly disclosed.

Example: common-neighbors prediction on a 5-person network



Five users (Alice, Bob, Carol, Dave, Eve). Rule: recommend $\{u, v\}$ if $S_{\text{CN}}[u, v] = |N(u) \cap N(v)| \geq \tau$, with $\tau = 2$.

Try it yourself first: for each non-edge, list mutual friends, count, threshold.

Neighbor sets: $N(A) = \{B, C\}$, $N(B) = \{A, C, D\}$, $N(C) = \{A, B, D\}$, $N(D) = \{B, C, E\}$, $N(E) = \{D\}$. The $\binom{5}{2} - 6 = 4$ non-edges score as follows:

Non-edge $\{u, v\}$	$N(u) \cap N(v)$	S_{CN}	Recommended?
$\{A, D\}$	$\{B, C\}$	2	**yes**
$\{A, E\}$	$\varnothing$	0	no
$\{B, E\}$	$\{D\}$	1	no
$\{C, E\}$	$\{D\}$	1	no

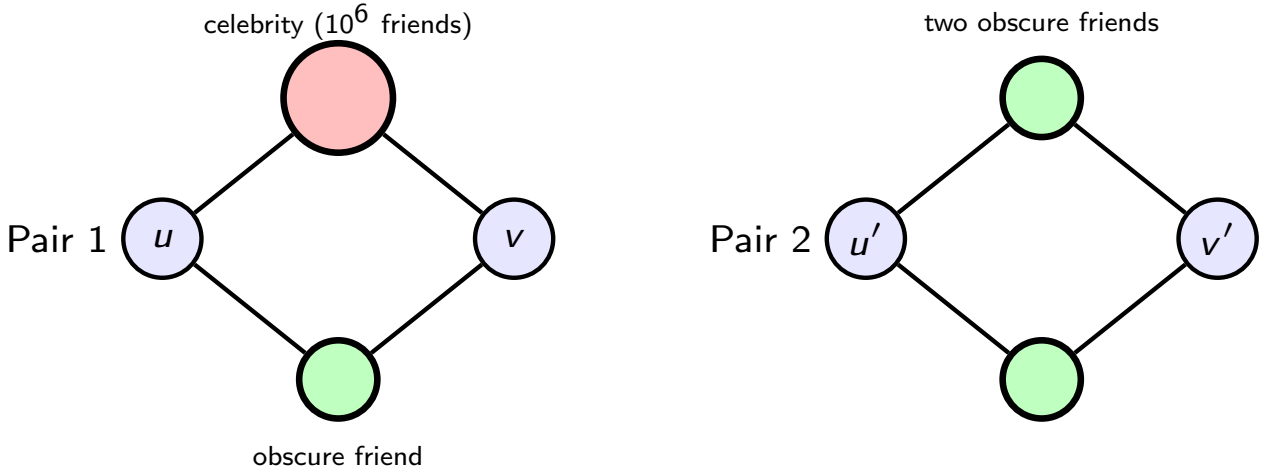
The system recommends *exactly one* future friendship: $\{A, D\}$. Intuitive: A is tightly connected to both B and C, who are both friends with D.

The threshold matters — and so does the celebrity problem (1/2)

Threshold sensitivity. Lowering the threshold to $\tau = 1$ expands the recommendations from $\{\{A, D\}\}$ to $\{\{A, D\}, \{B, E\}, \{C, E\}\}$ — now also suggesting E to B and to C on the basis of the single shared connection D.

The standard **precision–recall trade-off**: smaller $\tau \Rightarrow$ more recall, more false positives. Production systems tune τ on validation data.

Where common neighbors fall short. On a real social network of hundreds of millions of users, the count $|N(u) \cap N(v)|$ is too coarse. Imagine two pairs that each share *two* mutual friends:



The threshold matters — and so does the celebrity problem (2/2)

From the previous slide: two pairs (u, v) and (u', v'), each sharing two mutual friends.

S_{CN} assigns both pairs the same score, 2. But intuitively pair 2 is more informative — two close confidants are stronger evidence of a future friendship than one celebrity follow plus one confidant. The celebrity adds little: among their 10^6 followers, almost any pair will share them.

Two ways the literature has fixed this.

- **Normalize by endpoint degrees** d_u, d_v — so that "sharing 2 friends out of 200" counts more than "sharing 2 out of 20,000". Gives the *Sørensen*, *Salton*, and *Jaccard* indices.
- **Weight by common-neighbor degree** d_w — so that a low-degree common neighbor (rare overlap) counts for more than a high-degree one (everyone overlaps there). Gives the *Resource Allocation* and *Adamic–Adar* indices.

The next two subsections develop these in turn.

7 Neighborhood overlap detection (link prediction)

7.2 Endpoint-degree normalizations: Sørensen, Salton, Jaccard

Normalizing by endpoint degree

S_{CN} is biased toward high-degree pairs — a high-degree node has many neighbors that *could* be shared. The next three indices correct this by dividing by a function of (d_u, d_v).

Sørensen–Dice (twice intersection over sum of degrees):

$$S_{\mathrm{Sørensen}}[u, v] \;=\; \frac{2\,|N(u) \cap N(v)|}{d_u + d_v}.$$

Salton / cosine (cosine of 0/1-indicator vectors of $N(u)$, $N(v)$):

$$S_{\mathrm{Salton}}[u, v] \;=\; \frac{|N(u) \cap N(v)|}{\sqrt{d_u\, d_v}}.$$

Jaccard (intersection over union):

$$\boxed{S_{\mathrm{Jaccard}}[u, v] \;=\; \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}.}$$

All three live in $[0, 1]$ and suppress the high-degree bias of S_{CN}.

7 Neighborhood overlap detection (link prediction)

7.3 Common-neighbor degree weighting: RA vs. AA

Weighting by common-neighbor degree: RA and AA

A different route to suppressing celebrities: don't divide by endpoint degrees, but instead *down-weight common neighbors that are themselves high-degree hubs*.

Definition (Resource Allocation index).

$$S_{\mathrm{RA}}[u, v] = \sum_{w \in N(u) \cap N(v)} \frac{1}{d_w}.$$

Definition (Adamic–Adar index).

$$S_{\mathrm{AA}}[u, v] = \sum_{w \in N(u) \cap N(v)} \frac{1}{\log d_w}.$$

(Defined when every common neighbor has $d_w \geq 2$, so $\log d_w > 0$. The base of the log doesn't affect rankings — it's an overall positive scaling.)

Both use the same template $\sum_{w \in N(u) \cap N(v)} g(d_w)$ with g decreasing in d:

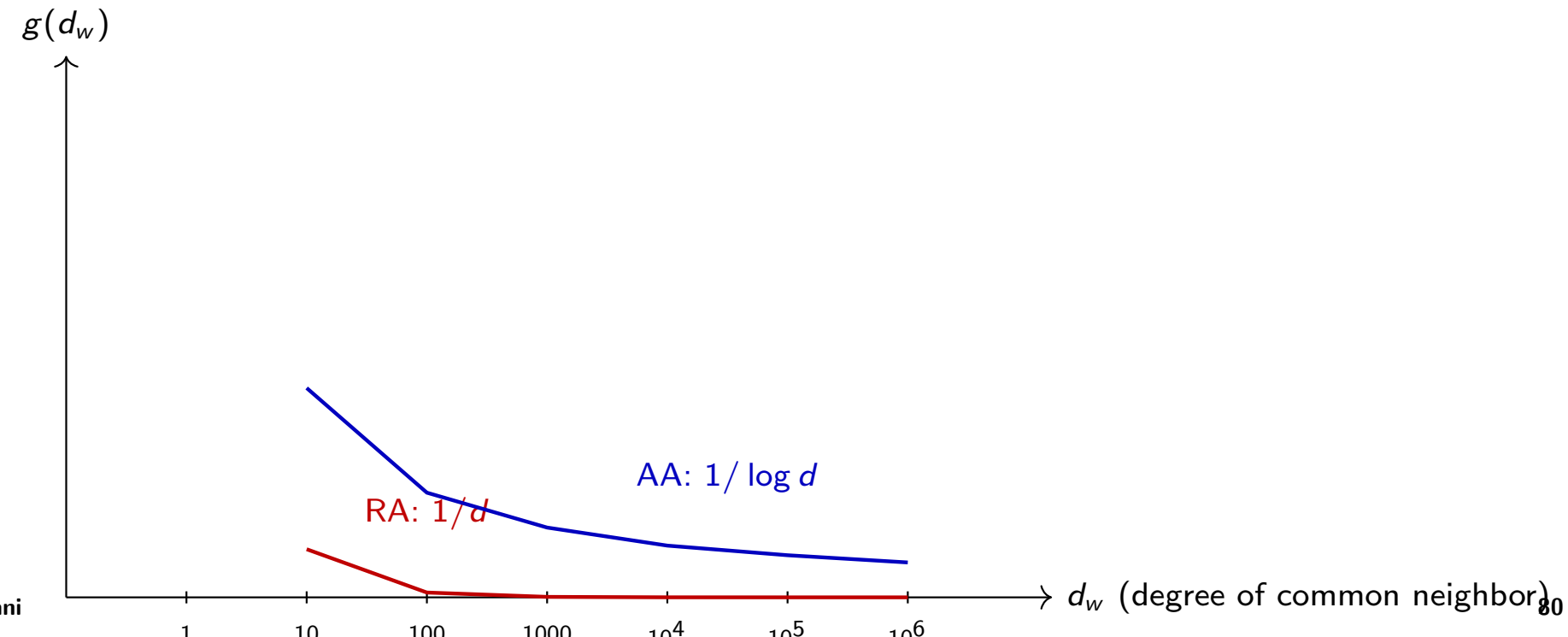
- RA picks $g(d) = 1/d$ (*linear* suppression of hubs).
- AA picks $g(d) = 1/\log d$ (*logarithmic*, gentler suppression).

RA vs. AA: aggressive vs. gentle suppression of hubs

Numerical effect. A common neighbor of degree $d_w = 1000$ contributes:

$$\text{to RA: } \frac{1}{1000} = 0.001, \qquad \text{to AA: } \frac{1}{\log 1000} \approx \frac{1}{6.9} \approx 0.145.$$

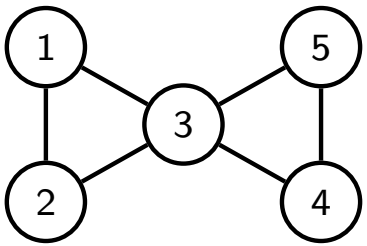
Two orders of magnitude apart. RA *essentially zeros out hubs*; AA *merely discounts them*.



7 Neighborhood overlap detection (link prediction)

7.4 All five measures on one example

Example: five local overlap measures on the bowtie



Bowtie: $V = \{1, 2, 3, 4, 5\}$, $E = \{12, 13, 23, 34, 35, 45\}$.
Degrees: $d_1 = d_2 = d_4 = d_5 = 2$, $d_3 = 4$.

Score the non-adjacent pair $\{1, 4\}$. $N(1) = \{2, 3\}$, $N(4) = \{3, 5\}$, so $N(1) \cap N(4) = \{3\}$ (size 1) and $N(1) \cup N(4) = \{2, 3, 5\}$ (size 3). Therefore (using natural log):

$$
\begin{aligned}
S_{\mathrm{CN}}[1,4] &= 1, \\
S_{\text{Sørensen}}[1,4] &= \frac{2 \cdot 1}{2+2} = \frac{1}{2}, \\
S_{\mathrm{Salton}}[1,4] &= \frac{1}{\sqrt{2 \cdot 2}} = \frac{1}{2}.
\end{aligned}
$$

$$
\begin{aligned}
S_{\mathrm{Jaccard}}[1,4] &= \frac{1}{3}, \\
S_{\mathrm{RA}}[1,4] &= \frac{1}{d_3} = \frac{1}{4}, \\
S_{\mathrm{AA}}[1,4] &= \frac{1}{\log 4} \approx 0.721.
\end{aligned}
$$

What local overlap can and cannot see

Limitation made vivid. On the same bowtie, the pair $\{1, 5\}$ also shares only node 3 as a common neighbor and has the same endpoint degrees as $\{1, 4\}$.

Pair	S_{CN}	$S_{\text{Sørensen}}$	S_{Salton}	S_{Jaccard}	S_{RA}	S_{AA}
$\{1, 4\}$	1	$1/2$	$1/2$	$1/3$	$1/4$	≈ 0.721
$\{1, 5\}$	1	$1/2$	$1/2$	$1/3$	$1/4$	≈ 0.721

Every local overlap measure assigns these two pairs identical scores. Yet $\{1, 4\}$ is at graph-distance 2 in the bowtie while $\{1, 5\}$ is at distance 2 as well — in this case the local measures are right.

But on a real graph, two pairs with no common neighbors at all ($S_{\text{CN}} = 0$) can be at very different graph distances — one perhaps reachable through a length-3 path, the other not at all. Local measures cannot tell them apart.

This motivates global overlap measures (next session) that aggregate over paths of every length: the Katz index, Leicht–Holme–Newman, and personalized PageRank — all of which have closed-form $(I - \kappa M)^{-1}$ expressions for an appropriate graph matrix M.

Summary: the pre-GNN toolkit, in one slide

Granularity	Feature	What it captures
Per-node (importance)	Degree	"How many friends?"
	Eigenvector centrality	"How important are my friends?"
	Katz	+ free baseline (works on DAGs)
	PageRank	+ divide by out-degree
	Closeness	"How quickly do I reach everyone?"
	Betweenness	"Am I a gatekeeper?"
Per-node (structure)	Degree	1-hop volume
	Clustering coefficient	triangle density at u
	Graphlet count vector	4-hop fingerprint, 73 orbit counts
Per-graph	Bag of nodes	histogram of node features
	WL kernel	color-refinement histogram
	Graphlet kernel	# induced subgraphs of each shape
Per-pair (overlap)	Common neighbors	$\lvert N(u) \cap N(v) \rvert$
	Sørensen, Salton	normalize by endpoint degrees
	Jaccard	$\lvert N(u) \cap N(v) \rvert / \lvert N(u) \cup N(v) \rvert$
	Resource Allocation	$\sum 1/d_w$ (kills celebrities)
	Adamic–Adar	$\sum 1/\log d_w$ (gentler)

Next session. The graph Laplacian, spectral clustering, and the limitations of hand-engineered features — the bridge to learned node embeddings.

Acknowledgements and references

Primary source. Hamilton, *Graph Representation Learning* (2020), Chapter 2.

Centrality measures. Newman, *Networks* (2018), Chapter 7.

Web-spam case study. Becchetti, Castillo, Donato, Leonardi, Baeza-Yates, "Link analysis for Web spam detection", *ACM TWEB* (2008).

Graphlet degree vector. Pržulj, "Biological network comparison using graphlet degree distribution", *Bioinformatics* (2007). Milenković & Pržulj, *Cancer Informatics* (2008).

GSN. Bouritsas, Frasca, Zafeiriou, Bronstein, "Improving graph neural network expressivity via subgraph isomorphism counting", *IEEE TPAMI* (2023).

Weisfeiler–Leman. Weisfeiler & Leman (1968). Shervashidze, Schweitzer, van Leeuwen, Mehlhorn, Borgwardt, "Weisfeiler–Lehman graph kernels", *JMLR* (2011).

Link prediction overlap measures. Adamic & Adar, *Social Networks* (2003); Zhou, Lü & Zhang, "Predicting missing links via local information", *EPJ B* (2009); see Hamilton (2020), §2.2 for a unified treatment.

Image sources are credited on individual slides where applicable.